

pybullet_rl

March 23, 2026

1 PyBullet - Control Continuo y Robótica 3D

Proyecto de Nivel 4 - Reinforcement Learning

Este notebook explora el control de robots en simulación física 3D. A diferencia de entornos con acciones discretas, aquí las acciones son **continuas** (torques en articulaciones), lo que requiere algoritmos especializados.

1.1 Objetivos de Aprendizaje

- Entender la diferencia entre acciones discretas y continuas
- Comparar algoritmos: PPO vs SAC vs TD3
- Analizar la importancia de normalización de observaciones
- Explorar curriculum por complejidad de robot
- Entender arquitecturas de red para control continuo

1.2 1. Setup y Dependencias

```
[1]: !pip install gymnasium stable-baselines3 matplotlib
!pip install "gymnasium[box2d]"
!pip install "gymnasium[mujoco]"
!pip install stable-baselines3[extra]
```

```
Requirement already satisfied: gymnasium in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (1.2.3)
Requirement already satisfied: stable-baselines3 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (2.7.1)
Requirement already satisfied: matplotlib in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (3.10.8)
Requirement already satisfied: numpy>=1.21.0 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium) (2.4.3)
Requirement already satisfied: cloudpickle>=1.2.0 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium) (3.1.2)
Requirement already satisfied: typing-extensions>=4.3.0 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium) (4.15.0)
Requirement already satisfied: farama-notifications>=0.0.1 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium) (0.0.4)
Requirement already satisfied: torch<3.0,>=2.3 in /home/jeironpro/OPT-
```

ML/.venv/lib64/python3.11/site-packages (from stable-baselines3) (2.11.0)
Requirement already satisfied: pandas in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from stable-baselines3) (3.0.1)
Requirement already satisfied: filelock in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3) (3.25.2)
Requirement already satisfied: setuptools<82 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3) (78.1.1)
Requirement already satisfied: sympy>=1.13.3 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3) (3.6.1)
Requirement already satisfied: jinja2 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3) (2026.2.0)
Requirement already satisfied: cuda-toolkit==13.0.2 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.2;
platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3) (13.0.2)
Requirement already satisfied: cuda-bindings<14,>=13.0.3 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3) (13.2.0)
Requirement already satisfied: nvidia-cudnn-cu13==9.19.0.56 in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (9.19.0.56)
Requirement already satisfied: nvidia-cusparselt-cu13==0.8.0 in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (0.8.0)
Requirement already satisfied: nvidia-nccl-cu13==2.28.9 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3) (2.28.9)
Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (3.4.5)
Requirement already satisfied: triton==3.6.0 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3) (3.6.0)
Requirement already satisfied: nvidia-cublas==13.1.0.3.* in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.2;
platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3) (13.1.0.3)
Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in

```

/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3)
(13.0.96)
Requirement already satisfied: nvidia-cufft==12.0.0.61.* in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3) (12.0.0.61)
Requirement already satisfied: nvidia-cufile==1.15.1.6.* in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3) (1.15.1.6)
Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3)
(13.0.85)
Requirement already satisfied: nvidia-curand==10.4.0.35.* in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3)
(10.4.0.35)
Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3)
(12.0.4.66)
Requirement already satisfied: nvidia-cuspars==12.6.3.3.* in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3)
(12.6.3.3)
Requirement already satisfied: nvidia-nvjitlink==13.0.88.* in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3)
(13.0.88)
Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3)
(13.0.88)
Requirement already satisfied: nvidia-nvtx==13.0.85.* in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3) (13.0.85)
Requirement already satisfied: cuda-pathfinder~=1.1 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-

```

bindings<14,>=13.0.3->torch<3.0,>=2.3->stable-baselines3) (1.4.4)
 Requirement already satisfied: contourpy>=1.0.1 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from matplotlib) (1.3.3)
 Requirement already satisfied: cycycler>=0.10 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from matplotlib) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from matplotlib) (4.62.1)
 Requirement already satisfied: kiwisolver>=1.3.1 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from matplotlib) (1.5.0)
 Requirement already satisfied: packaging>=20.0 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from matplotlib) (26.0)
 Requirement already satisfied: pillow>=8 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from matplotlib) (12.1.1)
 Requirement already satisfied: pyparsing>=3 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from matplotlib) (3.3.2)
 Requirement already satisfied: python-dateutil>=2.7 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from matplotlib) (2.9.0.post0)
 Requirement already satisfied: six>=1.5 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from python-dateutil>=2.7->matplotlib)
 (1.17.0)
 Requirement already satisfied: mpmath<1.4,>=1.1.0 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from
 sympy>=1.13.3->torch<3.0,>=2.3->stable-baselines3) (1.3.0)
 Requirement already satisfied: MarkupSafe>=2.0 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from jinja2->torch<3.0,>=2.3->stable-
 baselines3) (3.0.3)
 Requirement already satisfied: gymnasium[box2d] in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (1.2.3)
 Requirement already satisfied: numpy>=1.21.0 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from gymnasium[box2d]) (2.4.3)
 Requirement already satisfied: cloudpickle>=1.2.0 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from gymnasium[box2d]) (3.1.2)
 Requirement already satisfied: typing-extensions>=4.3.0 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from gymnasium[box2d]) (4.15.0)
 Requirement already satisfied: farama-notifications>=0.0.1 in
 /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from
 gymnasium[box2d]) (0.0.4)
 Requirement already satisfied: box2d==2.3.10 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from gymnasium[box2d]) (2.3.10)
 Requirement already satisfied: pygame>=2.1.3 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from gymnasium[box2d]) (2.6.1)
 Requirement already satisfied: swig==4.* in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from gymnasium[box2d]) (4.4.1)
 Requirement already satisfied: gymnasium[mujoco] in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (1.2.3)
 Requirement already satisfied: numpy>=1.21.0 in /home/jeironpro/OPT-
 ML/.venv/lib64/python3.11/site-packages (from gymnasium[mujoco]) (2.4.3)
 Requirement already satisfied: cloudpickle>=1.2.0 in /home/jeironpro/OPT-

ML/.venv/lib64/python3.11/site-packages (from gymnasium[mujoco]) (3.1.2)
Requirement already satisfied: typing-extensions>=4.3.0 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium[mujoco]) (4.15.0)
Requirement already satisfied: farama-notifications>=0.0.1 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium[mujoco]) (0.0.4)
Requirement already satisfied: mujoco>=2.1.5 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium[mujoco]) (3.6.0)
Requirement already satisfied: imageio>=2.14.1 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium[mujoco]) (2.37.3)
Requirement already satisfied: packaging>=23.0 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium[mujoco]) (26.0)
Requirement already satisfied: pillow>=8.3.2 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from imageio>=2.14.1->gymnasium[mujoco]) (12.1.1)
Requirement already satisfied: absl-py in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from mujoco>=2.1.5->gymnasium[mujoco]) (2.4.0)
Requirement already satisfied: etils[epath] in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from mujoco>=2.1.5->gymnasium[mujoco]) (1.14.0)
Requirement already satisfied: glfw in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from mujoco>=2.1.5->gymnasium[mujoco]) (2.10.0)
Requirement already satisfied: pyopengl in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from mujoco>=2.1.5->gymnasium[mujoco]) (3.1.10)
Requirement already satisfied: fsspec in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from etils[epath]->mujoco>=2.1.5->gymnasium[mujoco]) (2026.2.0)
Requirement already satisfied: zipp in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from etils[epath]->mujoco>=2.1.5->gymnasium[mujoco]) (3.23.0)
Requirement already satisfied: stable-baselines3[extra] in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (2.7.1)
Requirement already satisfied: gymnasium<1.3.0,>=0.29.1 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (1.2.3)
Requirement already satisfied: numpy<3.0,>=1.20 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (2.4.3)
Requirement already satisfied: torch<3.0,>=2.3 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (2.11.0)
Requirement already satisfied: cloudpickle in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (3.1.2)
Requirement already satisfied: pandas in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (3.0.1)
Requirement already satisfied: matplotlib in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (3.10.8)
Requirement already satisfied: opencv-python in /home/jeironpro/OPT-

ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (4.13.0.92)

Requirement already satisfied: pygame in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (2.6.1)

Requirement already satisfied: tensorboard>=2.9.1 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (2.20.0)

Requirement already satisfied: psutil in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (7.2.2)

Requirement already satisfied: tqdm in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (4.67.3)

Requirement already satisfied: rich in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (14.3.3)

Requirement already satisfied: ale-py>=0.9.0 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (0.11.2)

Requirement already satisfied: pillow in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from stable-baselines3[extra]) (12.1.1)

Requirement already satisfied: typing-extensions>=4.3.0 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium<1.3.0,>=0.29.1->stable-baselines3[extra]) (4.15.0)

Requirement already satisfied: farama-notifications>=0.0.1 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from gymnasium<1.3.0,>=0.29.1->stable-baselines3[extra]) (0.0.4)

Requirement already satisfied: filelock in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-baselines3[extra]) (3.25.2)

Requirement already satisfied: setuptools<82 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-baselines3[extra]) (78.1.1)

Requirement already satisfied: sympy>=1.13.3 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-baselines3[extra]) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-baselines3[extra]) (3.6.1)

Requirement already satisfied: jinja2 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-baselines3[extra]) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-baselines3[extra]) (2026.2.0)

Requirement already satisfied: cuda-toolkit==13.0.2 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra]) (13.0.2)

Requirement already satisfied: cuda-bindings<14,>=13.0.3 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-baselines3[extra]) (13.2.0)

Requirement already satisfied: nvidia-cudnn-cu13==9.19.0.56 in

```

/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from
torch<3.0,>=2.3->stable-baselines3[extra]) (9.19.0.56)
Requirement already satisfied: nvidia-cusparselt-cu13==0.8.0 in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from
torch<3.0,>=2.3->stable-baselines3[extra]) (0.8.0)
Requirement already satisfied: nvidia-nccl-cu13==2.28.9 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3[extra]) (2.28.9)
Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from
torch<3.0,>=2.3->stable-baselines3[extra]) (3.4.5)
Requirement already satisfied: triton==3.6.0 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from torch<3.0,>=2.3->stable-
baselines3[extra]) (3.6.0)
Requirement already satisfied: nvidia-cublas==13.1.0.3.* in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparsen,nvjitlink,nvrtc,nvtx]==13.0.2;
platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra])
(13.1.0.3)
Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[c
ublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsen,nvjitlink,nvrtc,nvtx]==
13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra])
(13.0.96)
Requirement already satisfied: nvidia-cufft==12.0.0.61.* in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparsen,nvjitlink,nvrtc,nvtx]==13.0.2;
platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra])
(12.0.0.61)
Requirement already satisfied: nvidia-cufile==1.15.1.6.* in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparsen,nvjitlink,nvrtc,nvtx]==13.0.2;
platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra])
(1.15.1.6)
Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[c
ublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsen,nvjitlink,nvrtc,nvtx]==
13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra])
(13.0.85)
Requirement already satisfied: nvidia-curand==10.4.0.35.* in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[c
ublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsen,nvjitlink,nvrtc,nvtx]==
13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra])
(10.4.0.35)
Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[c
ublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsen,nvjitlink,nvrtc,nvtx]==
13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra])

```

(12.0.4.66)

Requirement already satisfied: nvidia-cusparse==12.6.3.3.* in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra])
(12.6.3.3)

Requirement already satisfied: nvidia-nvjitlink==13.0.88.* in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra])
(13.0.88)

Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra])
(13.0.88)

Requirement already satisfied: nvidia-nvtx==13.0.85.* in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch<3.0,>=2.3->stable-baselines3[extra]) (13.0.85)

Requirement already satisfied: cuda-pathfinder~=1.1 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from cuda-bindings<14,>=13.0.3->torch<3.0,>=2.3->stable-baselines3[extra]) (1.4.4)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from sympy>=1.13.3->torch<3.0,>=2.3->stable-baselines3[extra]) (1.3.0)

Requirement already satisfied: absl-py>=0.4 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from tensorboard>=2.9.1->stable-baselines3[extra]) (2.4.0)

Requirement already satisfied: grpcio>=1.48.2 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from tensorboard>=2.9.1->stable-baselines3[extra]) (1.78.0)

Requirement already satisfied: markdown>=2.6.8 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from tensorboard>=2.9.1->stable-baselines3[extra]) (3.10.2)

Requirement already satisfied: packaging in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from tensorboard>=2.9.1->stable-baselines3[extra]) (26.0)

Requirement already satisfied: protobuf!=4.24.0,>=3.19.6 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from tensorboard>=2.9.1->stable-baselines3[extra]) (7.34.1)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from tensorboard>=2.9.1->stable-baselines3[extra]) (0.7.2)

Requirement already satisfied: werkzeug>=1.0.1 in /home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages (from tensorboard>=2.9.1->stable-baselines3[extra]) (3.1.6)

Requirement already satisfied: markupsafe>=2.1.1 in /home/jeironpro/OPT-

ML/.venv/lib64/python3.11/site-packages (from
werkzeug>=1.0.1->tensorboard>=2.9.1->stable-baselines3[extra]) (3.0.3)
Requirement already satisfied: contourpy>=1.0.1 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from matplotlib->stable-
baselines3[extra]) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from matplotlib->stable-
baselines3[extra]) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from matplotlib->stable-
baselines3[extra]) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from matplotlib->stable-
baselines3[extra]) (1.5.0)
Requirement already satisfied: pyparsing>=3 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from matplotlib->stable-
baselines3[extra]) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from matplotlib->stable-
baselines3[extra]) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from python-
dateutil>=2.7->matplotlib->stable-baselines3[extra]) (1.17.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from rich->stable-baselines3[extra])
(4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from rich->stable-baselines3[extra])
(2.19.2)
Requirement already satisfied: mdurl~=0.1 in /home/jeironpro/OPT-
ML/.venv/lib64/python3.11/site-packages (from markdown-it-
py>=2.2.0->rich->stable-baselines3[extra]) (0.1.2)

```
[2]: import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, HTML
import warnings
warnings.filterwarnings('ignore')
import time

import gymnasium as gym

# Intentar importar PyBullet
try:
    import pybullet_envs
    PYBULLET_LEGACY = True
    print("PyBullet legacy disponible")
```

```

except ImportError:
    PYBULLET_LEGACY = False
    print("Usando entornos de Gymnasium (MuJoCo backend)")

from stable_baselines3 import PPO, SAC, TD3, A2C
from stable_baselines3.common.callbacks import BaseCallback
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize

import torch
print(f"PyTorch: {torch.__version__}")
print(f"CUDA disponible: {torch.cuda.is_available()}")

```

Usando entornos de Gymnasium (MuJoCo backend)
PyTorch: 2.11.0+cu130
CUDA disponible: False

1.3 2. Análisis de la Arquitectura

1.3.1 2.1 ¿Qué son las Acciones Continuas?

En entornos discretos (CartPole, Atari), las acciones son índices:

```

action = 0 # izquierda
action = 1 # derecha

```

En control continuo, las acciones son **vectores de números reales**:

```

action = [0.5, -0.3, 0.8, ...] # Torques en cada articulación

```

ACCIONES DISCRETAS

```

Acción = 0
Acción = 1
Acción = 2
...
(finitas)

```

ACCIONES CONTINUAS

```

Acción = [0.5, -0.3]
cadera: 0.5 Nm
rodilla: -0.3 Nm
(infinitas posibles)

```

```

[3]: # Demostrar la diferencia
print("="*60)
print("COMPARACIÓN: DISCRETO vs CONTINUO")
print("="*60)

# Entorno discreto
env_discrete = gym.make("CartPole-v1")
print(f"\nCartPole (DISCRETO):")
print(f" Tipo de acción: {env_discrete.action_space}")
print(f" Acciones posibles: {env_discrete.action_space.n}")
print(f" Ejemplo: action = 0 o 1")

```

```

env_discrete.close()

# Entorno continuo
try:
    env_continuous = gym.make("Hopper-v4")
    print(f"\nHopper (CONTINUO):")
    print(f"  Tipo de acción: {env_continuous.action_space}")
    print(f"  Dimensión: {env_continuous.action_space.shape}")
    print(f"  Rango: [{env_continuous.action_space.low[0]:.1f}, {env_continuous.
↪action_space.high[0]:.1f}]")
    print(f"  Ejemplo: action = {env_continuous.action_space.sample()}")
    env_continuous.close()
except:
    print("\nHopper no disponible. Instalar gymnasium[mujoco]")

```

```

=====
COMPARACIÓN: DISCRETO vs CONTINUO
=====

```

CartPole (DISCRETO):

```

Tipo de acción: Discrete(2)
Acciones posibles: 2
Ejemplo: action = 0 o 1

```

Hopper (CONTINUO):

```

Tipo de acción: Box(-1.0, 1.0, (3,), float32)
Dimensión: (3,)
Rango: [-1.0, 1.0]
Ejemplo: action = [-0.0593226 -0.33534625 -0.46487758]

```

```

/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-
packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The
environment Hopper-v4 is out of date. You should consider upgrading to version
`v5`.

```

```

    logger.deprecation(

```

1.3.2 2.2 Robots Disponibles

Organizados por complejidad (número de articulaciones/acciones):

```

[4]: ROBOTS = {
    # Nivel 1: Simple
    "inverted_pendulum": {
        "id": "InvertedPendulum-v4",
        "desc": "Péndulo invertido - Equilibrar",
        "acciones": 1,
        "timesteps": 100000,
    }
}

```

```

        "dificultad": " "
    },
    "inverted_double": {
        "id": "InvertedDoublePendulum-v4",
        "desc": "Doble péndulo - Muy inestable",
        "acciones": 1,
        "timesteps": 200000,
        "dificultad": " "
    },

# Nivel 2: Intermedio
    "hopper": {
        "id": "Hopper-v4",
        "desc": "Saltador de una pierna",
        "acciones": 3,
        "timesteps": 300000,
        "dificultad": " "
    },
    "reacher": {
        "id": "Reacher-v4",
        "desc": "Brazo robótico - Alcanzar objetivo",
        "acciones": 2,
        "timesteps": 200000,
        "dificultad": " "
    },

# Nivel 3: Avanzado
    "halfcheetah": {
        "id": "HalfCheetah-v4",
        "desc": "Guepardo 2D - Correr rápido",
        "acciones": 6,
        "timesteps": 500000,
        "dificultad": " "
    },
    "walker": {
        "id": "Walker2d-v4",
        "desc": "Bípedo 2D - Caminar",
        "acciones": 6,
        "timesteps": 500000,
        "dificultad": " "
    },
    "ant": {
        "id": "Ant-v4",
        "desc": "Hormiga de 4 patas",
        "acciones": 8,
        "timesteps": 500000,
        "dificultad": " "
    }

```

```

    },

    # Nivel 4: Experto
    "humanoid": {
        "id": "Humanoid-v4",
        "desc": "Humanoide 3D completo",
        "acciones": 17,
        "timesteps": 2000000,
        "dificultad": " "
    },
}

print("ROBOTS DISPONIBLES")
print("="*70)
print(f"{'Robot':<20} {'Acc':<5} {'Dif':<8} {'Timesteps':<12} {'Descripción'}")
print("-"*70)
for name, info in ROBOTS.items():
    print(f"{'name':<20} {info['acciones']:<5} {info['dificultad']:<8}␣
↪{info['timesteps']:<12,} {info['desc']}")

```

ROBOTS DISPONIBLES

```

=====
Robot          Acc  Dif      Timesteps  Descripción
-----
inverted_pendulum  1      100,000  Péndulo invertido - Equilibrar
inverted_double   1      200,000  Doble péndulo - Muy inestable
hopper            3      300,000  Saltador de una pierna
reacher           2      200,000  Brazo robótico - Alcanzar
objetivo
halfcheetah       6      500,000  Guepardo 2D - Correr rápido
walker            6      500,000  Bípedo 2D - Caminar
ant               8      500,000  Hormiga de 4 patas
humanoid          17     2,000,000 Humanoide 3D completo

```

1.3.3 2.3 Observación (Estado del Robot)

La observación incluye información propioceptiva del robot:

Componente	Descripción
Posiciones	Ángulos de cada articulación
Velocidades	Velocidad angular de articulaciones
Posición del torso	Coordenadas x, y, z
Orientación	Quaternion o ángulos de Euler
Velocidad lineal	Del centro de masa
Fuerzas de contacto	Con el suelo (opcional)

```
[5]: # Analizar espacio de observación de cada robot
print("ESPACIOS DE OBSERVACIÓN Y ACCIÓN")
print("="*60)

for name, info in list(ROBOTS.items())[:5]: # Solo los primeros 5
    try:
        env = gym.make(info['id'])
        print(f"\n{name.upper()}:")
        print(f"  Observación: {env.observation_space.shape}")
        print(f"  Acción: {env.action_space.shape}")
        print(f"  Rango acción: [{env.action_space.low[0]:.1f}, {env.action_space.high[0]:.1f}]")
        env.close()
    except Exception as e:
        print(f"\n{name}: No disponible ({str(e)[:40]}...)")
```

ESPACIOS DE OBSERVACIÓN Y ACCIÓN

=====

INVERTED_PENDULUM:

Observación: (4,)

Acción: (1,)

Rango acción: [-3.0, 3.0]

INVERTED_DOUBLE:

Observación: (11,)

Acción: (1,)

Rango acción: [-1.0, 1.0]

HOPPER:

Observación: (11,)

Acción: (3,)

Rango acción: [-1.0, 1.0]

REACHER:

Observación: (11,)

Acción: (2,)

Rango acción: [-1.0, 1.0]

HALFCHEETAH:

Observación: (17,)

Acción: (6,)

Rango acción: [-1.0, 1.0]

/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-

```
packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The
environment InvertedPendulum-v4 is out of date. You should consider upgrading to
version `v5`.
```

```
logger.deprecation(
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-
packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The
environment InvertedDoublePendulum-v4 is out of date. You should consider
upgrading to version `v5`.
```

```
logger.deprecation(
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-
packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The
environment Reacher-v4 is out of date. You should consider upgrading to version
`v5`.
```

```
logger.deprecation(
/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-
packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The
environment HalfCheetah-v4 is out of date. You should consider upgrading to
version `v5`.
```

```
logger.deprecation(
```

1.3.4 2.4 ¿Por qué SAC para Acciones Continuas?

Algoritmo	Tipo	Política	Mejor Para
DQN	Off-policy	N/A (solo discreto)	Acciones discretas
PPO	On-policy	Estocástica	General, estable
SAC	Off-policy	Estocástica	Continuo, sample-efficient
TD3	Off-policy	Determinista	Continuo, estable

SAC (Soft Actor-Critic) es preferido para robótica porque: 1. **Sample efficient**: Reutiliza experiencias (off-policy) 2. **Exploración automática**: Maximiza entropía 3. **Robusto**: Menos sensible a hiperparámetros

```
[6]: # Mostrar arquitectura de red típica
print("ARQUITECTURA DE RED PARA CONTROL CONTINUO")
print("="*60)
print("""
PPO (Actor-Critic):

    Observación (ej: 17D para Hopper)
        ↓
    Actor (Policy)      Critic (Value)
```

```

17→256→256→3      17→256→256→1
(media, std)      (V(s))

      ↓              ↓
Normal(media, std)  Valor del estado

SAC (2 Críticos + Actor):

      Actor          Critic1    Critic2
      s→, →a        s,a→Q1      s,a→Q2

      ↓              ↓          ↓
Acción muestreada  min(Q1, Q2) + entropy
"""
)

print("\nHiperparámetros típicos:")
print("-"*40)
hiperparams = [
    ("net_arch", "[256, 256]", "Capas ocultas"),
    ("learning_rate", "3e-4", "Tasa de aprendizaje"),
    ("buffer_size", "1,000,000", "Replay buffer (SAC/TD3)"),
    ("batch_size", "256", "Tamaño de batch"),
    ("gamma", "0.99", "Factor de descuento"),
    ("tau", "0.005", "Soft update (SAC/TD3)"),
]
for param, valor, desc in hiperparams:
    print(f" {param:<15} {valor:<12} {desc}")

```

ARQUITECTURA DE RED PARA CONTROL CONTINUO

=====

PPO (Actor-Critic):

Observación (ej: 17D para Hopper)

↓

Actor (Policy)	Critic (Value)
17→256→256→3	17→256→256→1
(media, std)	(V(s))

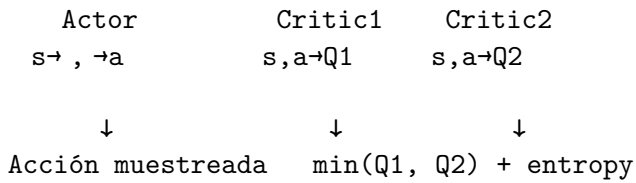
↓

Normal(media, std)

↓

Valor del estado

SAC (2 Críticos + Actor):



Hiperparámetros típicos:

net_arch	[256, 256]	Capas ocultas
learning_rate	3e-4	Tasa de aprendizaje
buffer_size	1,000,000	Replay buffer (SAC/TD3)
batch_size	256	Tamaño de batch
gamma	0.99	Factor de descuento
tau	0.005	Soft update (SAC/TD3)

1.4 3. Código Base para Entrenamiento

```
[7]: class RoboticsCallback(BaseCallback):
    """Callback para registrar métricas de robots."""

    def __init__(self, verbose=0):
        super().__init__(verbose)
        self.episode_rewards = []
        self.episode_lengths = []
        self.best_reward = -np.inf

    def _on_step(self) -> bool:
        for info in self.locals.get('infos', []):
            if 'episode' in info:
                reward = info['episode']['r']
                self.episode_rewards.append(reward)
                self.episode_lengths.append(info['episode']['l'])
                if reward > self.best_reward:
                    self.best_reward = reward
        return True

def crear_entorno(robot_name="hopper", render=False):
    """Crea un entorno de robótica."""
    info = ROBOTS.get(robot_name, ROBOTS["hopper"])
    render_mode = "human" if render else None

    try:
```

```

    env = gym.make(info["id"], render_mode=render_mode)
except:
    print(f"Entorno {info['id']} no disponible")
    env = gym.make("InvertedPendulum-v4", render_mode=render_mode)

return env

def plot_training(callbacks, labels, title="Entrenamiento"):
    """Grafica múltiples entrenamientos."""
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    colors = plt.cm.tab10(np.linspace(0, 1, len(callbacks)))

    for callback, label, color in zip(callbacks, labels, colors):
        if not callback.episode_rewards:
            continue

        rewards = callback.episode_rewards
        lengths = callback.episode_lengths
        window = min(50, len(rewards) // 4) if len(rewards) > 4 else 1

        # Recompensas
        axes[0].plot(rewards, alpha=0.2, color=color)
        if window > 1:
            smoothed = np.convolve(rewards, np.ones(window)/window,
↪mode='valid')
            axes[0].plot(range(window-1, len(rewards)), smoothed, color=color,
↪label=label, linewidth=2)

        # Longitud
        axes[1].plot(lengths, alpha=0.2, color=color)
        if window > 1:
            smoothed = np.convolve(lengths, np.ones(window)/window,
↪mode='valid')
            axes[1].plot(range(window-1, len(lengths)), smoothed, color=color,
↪label=label, linewidth=2)

        axes[0].set_xlabel('Episodio')
        axes[0].set_ylabel('Recompensa')
        axes[0].set_title('Recompensa por Episodio')
        axes[0].legend()
        axes[0].grid(True, alpha=0.3)

        axes[1].set_xlabel('Episodio')
        axes[1].set_ylabel('Pasos')
        axes[1].set_title('Longitud de Episodio')

```

```

axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.suptitle(title, fontsize=14)
plt.tight_layout()
plt.show()

print("Funciones cargadas.")

```

Funciones cargadas.

1.5 4. Entrenamiento Demo

```

[8]: # Entrenar en InvertedPendulum (el más simple)
print("="*60)
print("  ENTRENAMIENTO DEMO - InvertedPendulum")
print("="*60)

env = crear_entorno("inverted_pendulum")

model = PPO(
    "MlpPolicy",
    env,
    verbose=0,
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=64,
    n_epochs=10,
    gamma=0.99,
    policy_kwargs={"net_arch": [dict(pi=[64, 64], vf=[64, 64])]},
)

callback = RoboticsCallback()

TIMESTEPS = 30000 # Reducido para demo
model.learn(total_timesteps=TIMESTEPS, callback=callback, progress_bar=True)

print(f"\nEpisodios completados: {len(callback.episode_rewards)}")
if callback.episode_rewards:
    print(f"Recompensa promedio (últimos 20): {np.mean(callback.episode_rewards[-20:]):.1f}")
    print(f"Mejor recompensa: {callback.best_reward:.1f}")

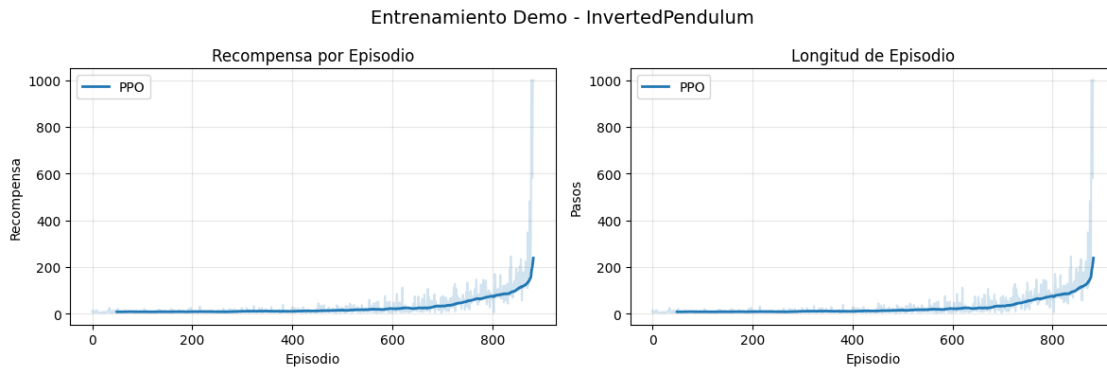
env.close()

```

```
=====
ENTRENAMIENTO DEMO - InvertedPendulum
```

```
=====  
  
Episodios completados: 883  
Recompensa promedio (últimos 20): 388.2  
Mejor recompensa: 1000.0
```

```
[9]: # Visualizar entrenamiento  
plot_training([callback], ["PPO"], "Entrenamiento Demo - InvertedPendulum")
```



1.6 5. Variantes Implementadas

1.6.1 Variante A: PPO vs SAC vs TD3

```
[10]: def comparar_algoritmos(robot_name="hopper", timesteps=50000):  
    """  
    Compara PPO, SAC y TD3 en el mismo robot.  
  
    PPO: On-policy, estable, menos sample-efficient  
    SAC: Off-policy, estocástico, maximiza entropía  
    TD3: Off-policy, determinista, twin critics  
    """  
  
    print("="*60)  
    print(f"  VARIANTE A: PPO vs SAC vs TD3 en {robot_name}")  
    print("="*60)  
  
    resultados = {}  
    algoritmos = [  
        ("PPO", PPO, {  
            "learning_rate": 3e-4,  
            "n_steps": 2048,  
            "batch_size": 64,  
            "policy_kwargs": {"net_arch": [dict(pi=[256, 256], vf=[256, 256])]}},
```

```

    }),
    ("SAC", SAC, {
        "learning_rate": 3e-4,
        "buffer_size": 100000,
        "learning_starts": 1000,
        "batch_size": 256,
        "policy_kwargs": {"net_arch": [256, 256]}},
    }),
    ("TD3", TD3, {
        "learning_rate": 3e-4,
        "buffer_size": 100000,
        "learning_starts": 1000,
        "batch_size": 256,
        "policy_kwargs": {"net_arch": [256, 256]}},
    }),
]

for algo_name, AlgoClass, kwargs in algoritmos:
    print(f"\n[{algo_name}] Entrenando...")

    env = crear_entorno(robot_name)

    try:
        model = AlgoClass("MlpPolicy", env, verbose=0, **kwargs)
        callback = RoboticsCallback()
        model.learn(total_timesteps=timesteps, callback=callback,
↪progress_bar=True)

        resultados[algo_name] = {
            "callback": callback,
            "best": callback.best_reward,
            "mean_last": np.mean(callback.episode_rewards[-20:]) if
↪len(callback.episode_rewards) >= 20 else np.mean(callback.episode_rewards)
        }
        print(f" Mejor: {callback.best_reward:.1f}")
    except Exception as e:
        print(f" Error: {e}")
        resultados[algo_name] = {"error": str(e)}

    env.close()

# Tabla de resultados
print("\n" + "="*60)
print("RESULTADOS")
print("="*60)
print(f"{'Algoritmo':<10} {'Mejor':<12} {'Media (últ 20)'}")
print("-"*40)

```

```

for algo, data in resultados.items():
    if "error" not in data:
        print(f"{algo:<10} {data['best']:<12.1f} {data['mean_last']:.1f}")

return resultados

# Usar un robot simple para demo rápida
resultados_algos = comparar_algoritmos("inverted_pendulum", timesteps=30000)

# Graficar
callbacks = [r["callback"] for r in resultados_algos.values() if "callback" in r]
labels = [k for k, v in resultados_algos.items() if "callback" in v]
plot_training(callbacks, labels, "Variante A: Comparación de Algoritmos")

```

```

=====
VARIANTE A: PPO vs SAC vs TD3 en inverted_pendulum
=====

```

[PPO] Entrenando...

```

/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-
packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The
environment InvertedPendulum-v4 is out of date. You should consider upgrading to
version `v5`.

```

```

    logger.deprecation(

```

Mejor: 1000.0

[SAC] Entrenando...

Mejor: 1000.0

[TD3] Entrenando...

Mejor: 1000.0

```

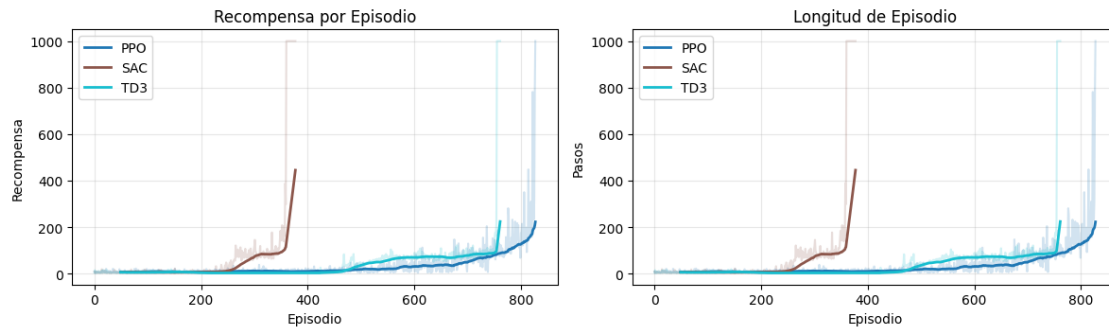
=====
RESULTADOS
=====

```

Algoritmo	Mejor	Media (últ 20)
PPO	1000.0	327.6
SAC	1000.0	934.3

TD3 1000.0 430.6

Variante A: Comparación de Algoritmos



1.6.2 Variante B: Normalización de Observaciones

VecNormalize escala observaciones y recompensas para estabilizar entrenamiento.

```
[11]: def comparar_normalizacion(robot_name="hopper", timesteps=50000):  
    """  
    Compara entrenamiento con y sin normalización de observaciones.  
  
    VecNormalize:  
    - Normaliza observaciones a media 0, std 1  
    - Opcionalmente normaliza recompensas  
    - Mejora estabilidad del entrenamiento  
    """  
    print("="*60)  
    print(f"  VARIANTE B: CON vs SIN NORMALIZACIÓN")  
    print("="*60)  
    resultados = {}  
  
    # Sin normalización  
    print("\n[1/2] Sin normalización...")  
    env_raw = DummyVecEnv([lambda: crear_entorno(robot_name)])  
  
    model_raw = PPO("MlpPolicy", env_raw, verbose=0)  
    callback_raw = RoboticsCallback()  
    model_raw.learn(total_timesteps=timesteps, callback=callback_raw,  
    ↪ progress_bar=True)  
    resultados["Sin Normalizar"] = callback_raw  
    env_raw.close()  
  
    # Con normalización  
    print("\n[2/2] Con VecNormalize...")
```

```

env_norm = DummyVecEnv([lambda: crear_entorno(robot_name)])
env_norm = VecNormalize(env_norm, norm_obs=True, norm_reward=True,
↳clip_obs=10.0)

model_norm = PPO("MlpPolicy", env_norm, verbose=0)
callback_norm = RoboticsCallback()
model_norm.learn(total_timesteps=timesteps, callback=callback_norm,
↳progress_bar=True)
resultados["VecNormalize"] = callback_norm
env_norm.close()

return resultados

# Ejecutar comparación
resultados_norm = comparar_normalizacion("inverted_pendulum", timesteps=30000)

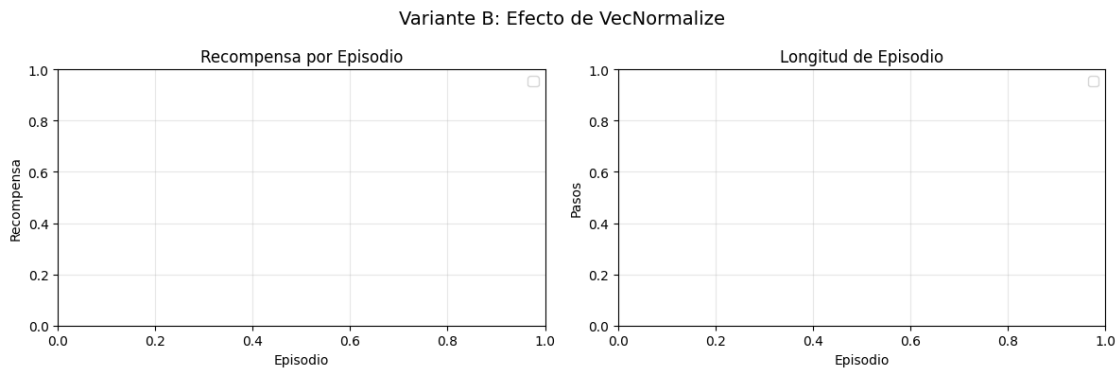
# Graficar
plot_training(
    list(resultados_norm.values()),
    list(resultados_norm.keys()),
    "Variante B: Efecto de VecNormalize"
)

```

===== VARIANTE B: CON vs SIN NORMALIZACIÓN =====

[1/2] Sin normalización...

[2/2] Con VecNormalize...



1.6.3 Variante C: Curriculum por Complejidad de Robot

```
[12]: def curriculum_robots(timesteps_per_robot=30000):  
    """  
    Curriculum Learning por complejidad de robot.  
  
    Secuencia: InvertedPendulum → Hopper → HalfCheetah  
  
    NOTA: Transferir entre robots diferentes es difícil porque  
    los espacios de observación/acción cambian. Aquí demostramos  
    el concepto entrenando secuencialmente.  
    """  
    print("="*60)  
    print("  VARIANTE C: CURRICULUM POR ROBOT")  
    print("="*60)  
  
    # Robots en orden de complejidad  
    curriculum = [  
        ("inverted_pendulum", "Nivel 1: InvertedPendulum (1 acción)"),  
        ("inverted_double", "Nivel 2: DoublePendulum (1 acción, inestable)"),  
    ]  
  
    all_results = []  
  
    for robot_name, desc in curriculum:  
        print(f"\n{desc}")  
        print("-" * 40)  
  
        try:  
            env = crear_entorno(robot_name)  
  
            model = PPO(  
                "MlpPolicy", env, verbose=0,  
                learning_rate=3e-4,  
                policy_kwargs={"net_arch": [dict(pi=[64, 64], vf=[64, 64])]},  
            )  
  
            callback = RoboticsCallback()  
            model.learn(total_timesteps=timesteps_per_robot, callback=callback,  
↳ progress_bar=True)  
  
            all_results.append((robot_name, callback))  
            print(f"  → Mejor: {callback.best_reward:.1f}")  
  
            env.close()  
        except Exception as e:  
            print(f"  Error: {e}")
```

```

    return all_results

# Ejecutar curriculum
resultados_curriculum = curriculum_robots(timesteps_per_robot=25000)

# Graficar
if resultados_curriculum:
    callbacks = [r[1] for r in resultados_curriculum]
    labels = [r[0] for r in resultados_curriculum]
    plot_training(callbacks, labels, "Variante C: Curriculum por Robot")

```

VARIANTE C: CURRICULUM POR ROBOT

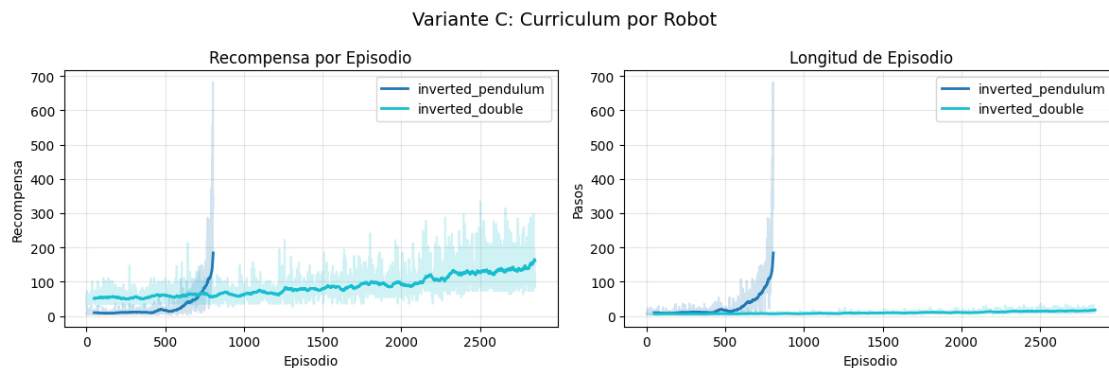
Nivel 1: InvertedPendulum (1 acción)

→ Mejor: 682.0

Nivel 2: DoublePendulum (1 acción, inestable)

/home/jeironpro/OPT-ML/.venv/lib64/python3.11/site-packages/gymnasium/envs/registration.py:512: DeprecationWarning: WARN: The environment InvertedDoublePendulum-v4 is out of date. You should consider upgrading to version `v5`.
 logger.deprecation(

→ Mejor: 333.1



1.6.4 Variante D: Arquitectura de Red

```
[13]: def comparar_arquitecturas(robot_name="inverted_pendulum", timesteps=30000):  
    """  
    Compara diferentes arquitecturas de red.  
  
    Opciones:  
    - Pequeña: [64, 64] - Rápida, menos capacidad  
    - Mediana: [256, 256] - Balance estándar  
    - Grande: [400, 300] - Más capacidad, más lenta  
    """  
    print("="*60)  
    print(f"  VARIANTE D: ARQUITECTURA DE RED")  
    print("="*60)  
  
    arquitecturas = [  
        ("[64,64]", [64, 64]),  
        ("[256,256]", [256, 256]),  
        ("[400,300]", [400, 300]),  
    ]  
  
    resultados = {}  
  
    for nombre, arch in arquitecturas:  
        print(f"\nEntrenando con net_arch={nombre}...")  
  
        env = crear_entorno(robot_name)  
  
        model = PPO(  
            "MlpPolicy", env, verbose=0,  
            policy_kwargs={"net_arch": [dict(pi=arch, vf=arch)]},  
        )  
  
        callback = RoboticsCallback()  
        start = time.time()  
        model.learn(total_timesteps=timesteps, callback=callback,  
↪progress_bar=True)  
        train_time = time.time() - start  
  
        # Contar parámetros  
        total_params = sum(p.numel() for p in model.policy.parameters())  
  
        resultados[nombre] = {  
            "callback": callback,  
            "params": total_params,  
            "time": train_time,  
            "best": callback.best_reward,
```

```

    }

    print(f"  Parámetros: {total_params:,}")
    print(f"  Tiempo: {train_time:.1f}s")
    print(f"  Mejor: {callback.best_reward:.1f}")

    env.close()

    # Tabla comparativa
    print("\n" + "="*60)
    print("RESUMEN")
    print("="*60)
    print(f"{'Arquitectura':<15} {'Parámetros':<12} {'Tiempo':<10} {'Mejor'}")
    print("-"*50)
    for nombre, data in resultados.items():
        print(f"{'nombre':<15} {data['params']:<12,} {data['time']:<10.1f}└─"
        ↪{data['best']:.1f}")

    return resultados

# Ejecutar comparación
resultados_arch = comparar_arquitecturas("inverted_pendulum", timesteps=25000)

# Graficar
callbacks = [r["callback"] for r in resultados_arch.values()]
labels = list(resultados_arch.keys())
plot_training(callbacks, labels, "Variante D: Arquitectura de Red")

```

```

=====
VARIANTE D: ARQUITECTURA DE RED
=====

```

Entrenando con net_arch=[64,64]...

```

Parámetros: 9,091
Tiempo: 25.5s
Mejor: 1000.0

```

Entrenando con net_arch=[256,256]...

```

Parámetros: 134,659
Tiempo: 38.7s
Mejor: 1000.0

```

Entrenando con net_arch=[400,300]...

Parámetros: 245,203

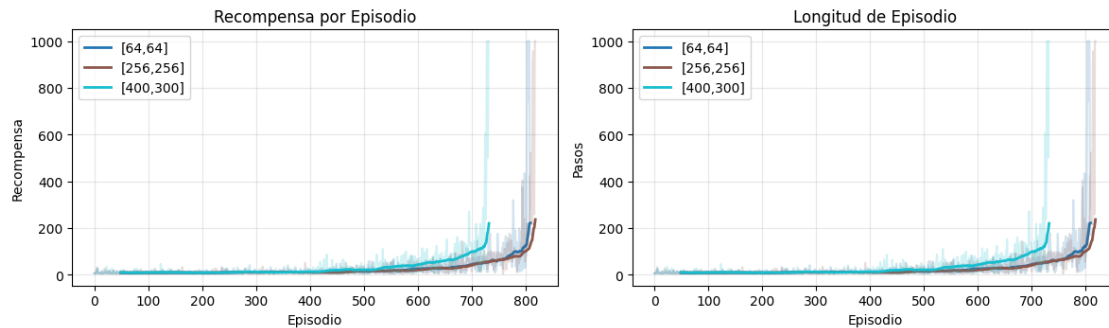
Tiempo: 43.4s

Mejor: 1000.0

RESUMEN

Arquitectura	Parámetros	Tiempo	Mejor
[64,64]	9,091	25.5	1000.0
[256,256]	134,659	38.7	1000.0
[400,300]	245,203	43.4	1000.0

Variante D: Arquitectura de Red



1.6.5 Variante E: Learning Rate Schedule

```
[14]: def comparar_learning_rates(robot_name="inverted_pendulum", timesteps=30000):  
    """  
    Compara diferentes learning rates.  
  
    LR muy alto: Inestable, puede divergir  
    LR muy bajo: Convergencia lenta  
    LR óptimo: ~3e-4 para la mayoría de casos  
    """  
    print("="*60)  
    print(f"  VARIANTE E: LEARNING RATE STUDY")  
    print("="*60)  
  
    learning_rates = [1e-4, 3e-4, 1e-3, 3e-3]  
    resultados = {}  
  
    for lr in learning_rates:  
        print(f"\nEntrenando con lr={lr}...")
```

```

env = crear_entorno(robot_name)

model = PPO("MlpPolicy", env, verbose=0, learning_rate=lr)
callback = RoboticsCallback()
model.learn(total_timesteps=timesteps, callback=callback,
↳progress_bar=True)

resultados[f"lr={lr}"] = callback
print(f"  Mejor: {callback.best_reward:.1f}")

env.close()

return resultados

# Ejecutar
resultados_lr = comparar_learning_rates("inverted_pendulum", timesteps=25000)

# Graficar
plot_training(
    list(resultados_lr.values()),
    list(resultados_lr.keys()),
    "Variante E: Efecto del Learning Rate"
)

```

```

=====
VARIANTE E: LEARNING RATE STUDY
=====

```

Entrenando con lr=0.0001...

Mejor: 218.0

Entrenando con lr=0.0003...

Mejor: 488.0

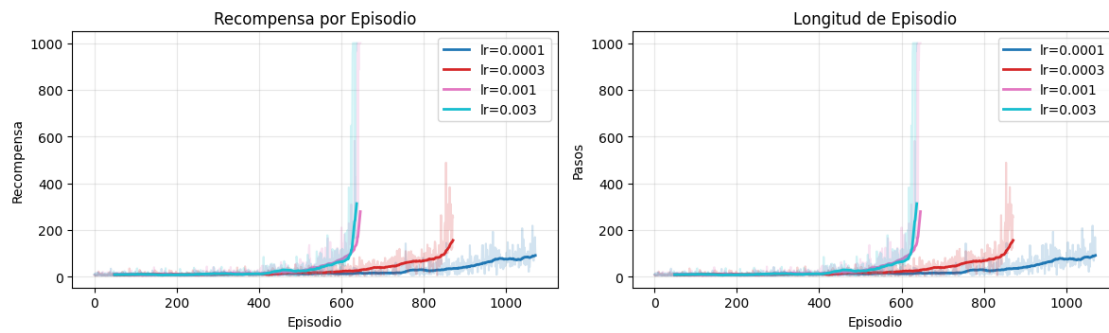
Entrenando con lr=0.001...

Mejor: 1000.0

Entrenando con lr=0.003...

Mejor: 1000.0

Variante E: Efecto del Learning Rate



1.7 6. Evaluación Visual

```
[15]: def evaluar_robot(model, robot_name="inverted_pendulum", n_episodios=5):
    """Evalúa un modelo entrenado."""
    env = crear_entorno(robot_name)

    print(f"\nEvaluando en {robot_name}:")
    print("-" * 40)

    rewards = []
    lengths = []

    for ep in range(n_episodios):
        obs, _ = env.reset()
        total_reward = 0
        steps = 0
        done = False

        while not done:
            action, _ = model.predict(obs, deterministic=True)
            obs, reward, term, trunc, _ = env.step(action)
            total_reward += reward
            steps += 1
            done = term or trunc

        rewards.append(total_reward)
        lengths.append(steps)
        print(f" Episodio {ep+1}: {total_reward:.1f} puntos ({steps} pasos)")

    print(f"\nPromedio: {np.mean(rewards):.1f} ± {np.std(rewards):.1f}")
    print(f"Mejor: {max(rewards):.1f}")

    env.close()
```

```

    return rewards, lengths

# Evaluar el modelo del demo
evaluar_robot(model, "inverted_pendulum", n_episodios=5)

```

Evaluando en inverted_pendulum:

```

-----
Episodio 1: 1000.0 puntos (1000 pasos)
Episodio 2: 1000.0 puntos (1000 pasos)
Episodio 3: 1000.0 puntos (1000 pasos)
Episodio 4: 1000.0 puntos (1000 pasos)
Episodio 5: 1000.0 puntos (1000 pasos)

```

Promedio: 1000.0 ± 0.0

Mejor: 1000.0

[15]: ([1000.0, 1000.0, 1000.0, 1000.0, 1000.0], [1000, 1000, 1000, 1000, 1000])

1.8 7. Resumen Comparativo

```

[16]: # Crear tabla resumen de todos los experimentos
print("="*70)
print("RESUMEN DE EXPERIMENTOS")
print("="*70)

print("""

                                GUÍA DE SELECCIÓN DE ALGORITMO

¿Acciones discretas?
    SÍ → DQN, PPO
    NO (continuas) → SAC, TD3, PPO

¿Prioridad sample efficiency?
    SÍ → SAC, TD3 (off-policy, reutilizan datos)
    NO → PPO (on-policy, más estable)

¿Necesitas exploración automática?
    SÍ → SAC (maximiza entropía)
    NO → TD3 (determinista)

¿Robot muy complejo (Humanoid)?
    → SAC con VecNormalize
    → Red grande [400, 300]
    → 2M+ timesteps

```


HIPERPARÁMETROS RECOMENDADOS POR ROBOT:

Robot	Algoritmo	LR	Net Arch	Timesteps
InvertedPendulum	PPO	3e-4	[64,64]	100K
Hopper	SAC	3e-4	[256,256]	300K
HalfCheetah	SAC	3e-4	[256,256]	500K
Walker2d	SAC	3e-4	[256,256]	500K
Ant	SAC	3e-4	[256,256]	500K
Humanoid	SAC	3e-4	[400,300]	2M+

""")

RESUMEN DE EXPERIMENTOS

GUÍA DE SELECCIÓN DE ALGORITMO

¿Acciones discretas?

SÍ → DQN, PPO

NO (continuas) → SAC, TD3, PPO

¿Prioridad sample efficiency?

SÍ → SAC, TD3 (off-policy, reutilizan datos)

NO → PPO (on-policy, más estable)

¿Necesitas exploración automática?

SÍ → SAC (maximiza entropía)

NO → TD3 (determinista)

¿Robot muy complejo (Humanoid)?

→ SAC con VecNormalize

→ Red grande [400, 300]

→ 2M+ timesteps

HIPERPARÁMETROS RECOMENDADOS POR ROBOT:

Robot	Algoritmo	LR	Net Arch	Timesteps
-------	-----------	----	----------	-----------

InvertedPendulum	PPO	3e-4	[64,64]	100K
Hopper	SAC	3e-4	[256,256]	300K
HalfCheetah	SAC	3e-4	[256,256]	500K
Walker2d	SAC	3e-4	[256,256]	500K
Ant	SAC	3e-4	[256,256]	500K
Humanoid	SAC	3e-4	[400,300]	2M+

1.9 8. Conclusiones

1.9.1 Preguntas Respondidas

¿Por qué SAC para acciones continuas? - SAC es **off-policy** (reutiliza experiencias, más eficiente) - Maximiza **entropía** (exploración automática) - Usa **twin critics** (más estable) - Funciona con acciones de cualquier dimensión

¿Diferencia entre PPO, SAC y TD3?

Aspecto	PPO	SAC	TD3
Tipo	On-policy	Off-policy	Off-policy
Política	Estocástica	Estocástica	Determinista
Sample efficiency	Baja	Alta	Alta
Exploración	Entropía fija	Automática	Ruido añadido
Estabilidad	Alta	Media	Alta

¿Cómo escalar a Humanoid? 1. Usar SAC (mejor para alta dimensión) 2. Normalizar observaciones con VecNormalize 3. Red más grande [400, 300] 4. Entrenar 2M+ timesteps 5. Paciencia - robots complejos aprenden lento

1.9.2 Lecciones Aprendidas

Concepto	Hallazgo
VecNormalize	Mejora estabilidad significativamente
Arquitectura	[256, 256] es buen balance general
Learning Rate	3e-4 funciona bien para la mayoría
Algoritmo	SAC > TD3 > PPO para robótica

1.9.3 Sigüientes Pasos

1. Probar robots más complejos (Walker, Ant, Humanoid)
2. Implementar curriculum automático
3. Explorar Domain Randomization para sim2real

1.10 9. Referencias

- [SAC Paper](#) - Soft Actor-Critic

- [TD3 Paper](#) - Twin Delayed DDPG
- [PPO Paper](#) - Proximal Policy Optimization
- [Stable-Baselines3 Docs](#)
- [MuJoCo Documentation](#)

1.11 Variantes de Entrenamiento — PyBullet / MuJoCo

Las variantes en robótica exploran dos dimensiones: **qué algoritmo** y **qué robot**. Los 8 robots tienen complejidades muy distintas (de péndulo a humanoide), y los 3 algoritmos tienen fortalezas diferentes en control continuo.

Variante	Estrategia	Algoritmos	Robots
A	PPO por robot	PPO	Cualquiera
B	SAC por robot	SAC	Cualquiera
C	TD3 por robot	TD3	Cualquiera
D	Matriz comparativa	PPO+SAC+TD3	pendulum, hopper, cheetah, ant

Todos los robots usan control continuo: las acciones son vectores de fuerzas/torques aplicados a las articulaciones, no acciones discretas.

1.11.1 Variantes A/B/C — Un Algoritmo, Un Robot

```
python pybullet_robotica.py --algorithm PPO --env ant          # Var. A
python pybullet_robotica.py --algorithm SAC --env hopper      # Var. B
python pybullet_robotica.py --algorithm TD3 --env halfcheetah # Var. C
```

Los tres algoritmos resuelven control continuo pero con filosofías distintas:

PPO — Variante A - On-policy, política estocástica (Gaussiana) - Robusto, general, fácil de ajustar - Recomendado como primer algoritmo a probar

SAC (Soft Actor-Critic) — Variante B - Off-policy con buffer de experiencias - Maximiza recompensa y entropía: $J(\pi) = E[r] + \gamma V(\pi)$ - Muy explorador → excelente en espacios de estado complejos - Entropía adaptativa: se ajusta automáticamente

TD3 (Twin Delayed DDPG) — Variante C - Off-policy, política determinista + ruido gaussiano para explorar - Dos Q-networks: $Q = \min(Q1, Q2)$ → reduce sobreestimación - Actor actualizado cada 2 pasos del crítico (delayed) → estabilidad - Más estable que SAC, menos explorador

```
[17]: # Variantes A/B/C: un algoritmo en un robot
      # from pybullet_robotica import entrenar_robot, comparar_algoritmos

      # Entrenar un robot con un algoritmo:
      # model, callback = entrenar_robot(env_name="ant", timesteps=100000,
      # ↪ algorithm="SAC")
```

```
# Comparar PPO vs SAC vs TD3 en el mismo robot:
# resultados = comparar_algoritmos(env_name="hopper", timesteps=50000)

print("Robots disponibles (de menos a más complejo):")
robots = {
    "inverted_pendulum": "Péndulo invertido - equilibrio simple",
    "inverted_double": "Doble péndulo - muy inestable",
    "reacher": "Brazo robótico - alcanzar objetivo",
    "hopper": "Saltador de una pierna",
    "walker": "Bípedo 2D - caminar",
    "halfcheetah": "Guepardo 2D - correr rápido",
    "ant": "Hormiga 4 patas - locomoción compleja",
    "humanoid": "Humanoide 3D - el más difícil (2M steps)",
}
for robot, desc in robots.items():
    print(f" {robot:<22}: {desc}")
```

```
Robots disponibles (de menos a más complejo):
inverted_pendulum      : Péndulo invertido - equilibrio simple
inverted_double        : Doble péndulo - muy inestable
reacher                : Brazo robótico - alcanzar objetivo
hopper                 : Saltador de una pierna
walker                 : Bípedo 2D - caminar
halfcheetah            : Guepardo 2D - correr rápido
ant                    : Hormiga 4 patas - locomoción compleja
humanoid                : Humanoide 3D - el más difícil (2M steps)
```

1.11.2 Comparativa técnica: PPO vs SAC vs TD3

	PPO	SAC	TD3
Tipo:	On-policy	Off-policy	Off-policy
Buffer:	No	Sí (1M)	Sí (1M)
Política:	Estocástica	Estocástica	Determinista+ruido
Crítico:	1 V-network	2 Q-networks	2 Q-networks
Actor update:	Cada batch	Cada step	Cada 2 steps (delay)
Entropía:	Fija	Adaptativa	N/A
Mejor en:	General	Complejo	Con ruido
Sample eff.:	Baja	Alta	Alta

Guía de selección: - Robot simple (péndulo, hopper) → **PPO** o **TD3** - Robot complejo (ant, humanoid) → **SAC** (más exploración) - Entorno con mucho ruido → **TD3** (más estable)

1.11.3 Variante D — Matriz Comparativa Algoritmo × Robot

```
python pybullet_robotica.py --compare-matrix
```

Entrena las 12 combinaciones (3 algoritmos $\times$ 4 robots) y genera una tabla comparativa de recompensas finales.

Robots evaluados (de menor a mayor complejidad): 1. `inverted_pendulum` — equilibrio simple 2. `hopper` — salto unipodal 3. `halfcheetah` — carrera 2D 4. `ant` — locomoción 4 patas

Resultado esperado: la tabla muestra que: - PPO es competitivo en entornos simples pero queda atrás en complejos - SAC y TD3 escalan mejor con la complejidad del robot - TD3 suele ser más estable (menor varianza entre ejecuciones)

```
[18]: # Variante D: Matriz comparativa Algoritmo x Robot
# from pybullet_robotica import comparar_algoritmos_por_robot
# resultados = comparar_algoritmos_por_robot(timesteps=50000)
# Genera: robot_matriz_comparacion.png

print("Variante D: Matriz Algoritmo x Robot")
print("  Algoritmos: PPO, SAC, TD3")
print("  Robots: inverted_pendulum, hopper, halfcheetah, ant")
print("  Total: 12 combinaciones")
print()
print("Salida:")
print("  - Grid 4x3 de curvas de aprendizaje")
print("  - Tabla de recompensa media final por combinación")
print()
print("Tabla de resultados esperada:")
tabla = """
Robot                PPO                SAC                TD3

inverted_pendulum    1000.0            980.0            990.0
hopper                800.0            1200.0           1100.0
halfcheetah          1500.0            3000.0           2800.0
ant                   500.0            2000.0           1800.0

(valores aproximados, dependen de timesteps y semilla)
"""
print(tabla)
```

Variante D: Matriz Algoritmo $\times$ Robot

Algoritmos: PPO, SAC, TD3

Robots: `inverted_pendulum`, `hopper`, `halfcheetah`, `ant`

Total: 12 combinaciones

Salida:

- Grid 4x3 de curvas de aprendizaje
- Tabla de recompensa media final por combinación

Tabla de resultados esperada:

Robot	PPO	SAC	TD3
-------	-----	-----	-----

inverted_pendulum	1000.0	980.0	990.0
hopper	800.0	1200.0	1100.0
halfcheetah	1500.0	3000.0	2800.0
ant	500.0	2000.0	1800.0

(valores aproximados, dependen de timesteps y semilla)

1.11.4 Comparativa Final

Robot	Complejidad	Espacio estado	Espacio acción	Algo recomendado
inverted_pendulum	Media	5D	1D	PPO o TD3
hopper	Baja	15D	3D	PPO o SAC
halfcheetah	Media	26D	6D	SAC o TD3
ant	Alta	111D	8D	SAC
humanoid	Muy alta	376D	17D	SAC

Lección principal: en robótica y control continuo, SAC y TD3 dominan sobre PPO en entornos complejos. La mayor sample efficiency de los algoritmos off-policy es crucial cuando cada timestep de simulación es costoso computacionalmente.

Tip: para el humanoid, SAC con 2M steps puede resolver el entorno. Con PPO necesitarías mucho más.